A Mini Project Report on

AI-Powered Business Finance and Inventory Analysis System

Finance Management · Inventory Analytics · Demand Forecasting · Expense Anomaly Detection

Submitted in partial fulfilment of the requirements of

SEWDL Lab

TY B.Tech — Information Technology, Semester V

Submitted by

Prathamesh Salunkhe (Roll No: 24101C0021)
Omkar Padhi (Roll No: 24101C0003)
Atharva Chavan (Roll No: 24101C0006)
Aryan Kelkar (Roll No: 24101C0020)

Under the guidance of

Prof. Debarati Ghosal

Department of Information Technology
Vidyalankar Institute of Technology
Wadala (E), Mumbai – 400037
(Affiliated to University of Mumbai)

Academic Year 2026–2027

CERTIFICATE

This is to certify that the mini project entitled

“AI-Powered Business Finance and Inventory Analysis System”

is a bonafide work done by

Prathamesh Salunkhe (24101C0021)
Omkar Padhi (24101C0003)
Atharva Chavan (24101C0006)
Aryan Kelkar (24101C0020)

and is submitted in partial fulfilment of the requirements of SEWDL Lab, TY B.Tech Information Technology, Semester V, at Vidyalankar Institute of Technology, Wadala, Mumbai, during the academic year 2026–2027.

Prof. Debarati Ghosal **Project Guide**	**Head of the Department**	**Principal**

Date:
Place: Mumbai

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Prof. Debarati Ghosal**, for her guidance, encouragement and valuable feedback throughout this project. We thank the Head of the Department of Information Technology and the Principal of Vidyalankar Institute of Technology for providing the facilities and support needed to carry out this work.

We are also grateful to the faculty members and laboratory staff of the department for their help, and to our families and friends for their constant encouragement during the semester.

Prathamesh Salunkhe (24101C0021)
Omkar Padhi (24101C0003)
Atharva Chavan (24101C0006)
Aryan Kelkar (24101C0020)

ABSTRACT

Small and medium enterprises (SMEs) usually keep sales, expenses and stock in separate registers or spreadsheets. As a result the owner cannot see at a glance whether the business is profitable, why cash is short in a month that shows a profit, which products are about to run out, or which expenses are unusual. This project presents the **AI-Powered Business Finance and Inventory Analysis System**, a web-based business-intelligence platform that brings these views together in one place.

The system is built with Python and Flask on a MySQL database, with an automatic SQLite fallback so that it can always be demonstrated. A demo dataset covering twelve months (October 2025 – September 2026) is generated deterministically: 24 products in 8 categories, 20 customers, 6 staff, 4,828 sales orders with 9,448 order lines and 253 expenses in 8 categories. An analytics engine computes revenue, cost of goods sold, gross and net profit, margins, cash inflow and outflow, inventory value, stock status and sales performance directly from the database.

Four intelligent features are implemented on top of these analytics: (1) **demand forecasting** that fits a scikit-learn linear-regression trend on seasonally adjusted monthly sales and recommends stock levels with a safety buffer; (2) **expense anomaly detection** using a robust (median / MAD) Z-score and month-over-month spike rules; (3) a deterministic **Financial Health Score** out of 100 built from five measured factors; and (4) a rule-based **AI recommendation engine** that turns these results into prioritised actions. A customer panel allows customers to register, browse products and place orders, which immediately update stock and analytics.

On the demo data the system reports revenue of ₹2.07 crore, net profit of ₹25.59 lakh (12.3% net margin), a Financial Health Score of 76/100, seven products at or below their minimum stock and four planted unusual expenses, all of which are detected. The average back-test accuracy of the demand forecast is 73% across the 24 products.

Keywords: business intelligence, SME finance, inventory management, demand forecasting, anomaly detection, robust Z-score, financial health score, Flask, MySQL, scikit-learn.

TABLE OF CONTENTS

1. Introduction 1
1.1 Background 1
1.2 Motivation 1
1.3 Scope of the Work 1
2. Problem Statement and Objectives 1
3. Review of Existing Approaches 2
4. Requirements Specification 2
4.1 Functional Requirements 2
4.2 Non-Functional Requirements 2
5. Proposed System and Architecture 3
6. System Design (UML Diagrams) 4
6.1 Use Case and Activity Diagrams 4
6.2 Class Diagram 4
6.3 Sequence Diagrams 5
6.4 Entity–Relationship Diagram 6
7. Database Design and Dataset 6
7.1 Tables 6
7.2 Demo Dataset 6
8. Analytics and AI Methodology 7
8.1 Finance Metrics 7
8.2 Demand Forecasting 7
8.3 Expense Anomaly Detection 8
8.4 Financial Health Score 8
8.5 AI Recommendation Engine 8
9. Implementation and Modules 9
10. Testing 9
11. Results and Discussion 10
11.1 Financial Results 10
11.2 Inventory and Forecast Results 10
11.3 Anomaly Detection Results 11
11.4 Financial Health Score 11
11.5 Limitations 11
12. Conclusion and Future Scope 12
12.1 Conclusion 12
12.2 Future Scope 12
References 12

LIST OF FIGURES

Figure 1: System Architecture
Figure 2: Use Case Diagram and Activity Diagram (Demand Forecast)
Figure 3: Class Diagram
Figure 4: Sequence Diagram — Customer Places an Order
Figure 5: Entity–Relationship Diagram
Figure 6: Key Screens of the Application

LIST OF TABLES

Table 1: Functional requirements
Table 2: Database tables
Table 3: Summary of the demo dataset
Table 4: Finance formulas used
Table 5: Financial Health Score factors (20 points each)
Table 6: Test cases
Table 7: Monthly profit and cash flow (₹)
Table 8: Unusual expenses detected
Table 9: Financial Health Score on the demo data

1. Introduction

1.1 Background

Every business records three kinds of facts: what it sells, what it spends and what it holds in stock. Large companies use ERP and business-intelligence suites to combine these facts, but most SMEs in India still rely on bill books, a separate accounting package and spreadsheets for stock. The data exists, but it is not connected, so the owner has to calculate profit, cash position and re-order needs manually, usually at the end of the month when it is too late to act.

1.2 Motivation

The original version of this project, “Smart Retail Analytics”, was an e-commerce storefront with a small admin dashboard and only ten sample orders. We transformed it into a decision-support system for the business owner. The motivation was to answer practical questions directly from the data: Is the business healthy? Why did cash fall in a profitable month? Which products will run out next month? Which expenses look wrong? What should be done first?

1.3 Scope of the Work

- Finance: revenue, cost of goods sold (COGS), gross profit, operating expenses, net profit, margins, growth and cash flow.
- Expenses: recording, categorising, filtering, CSV export and automatic detection of unusual expenses.
- Inventory: stock levels, minimum levels, inventory value, stock status, low-stock and out-of-stock alerts, restocking.
- Sales: top-selling and slow-moving products, category performance and demand trends.
- Intelligence: demand forecasting, a Financial Health Score and AI business recommendations.
- Customer panel: registration, product catalogue, cart, checkout and order tracking, with staff-side order management.

2. Problem Statement and Objectives

Problem statement: *to design and implement a web-based system that combines the sales, expense and inventory records of a small or medium business, computes its financial and inventory position automatically, and uses forecasting, anomaly detection and rule-based reasoning to tell the owner what needs attention.*

The objectives of the project are:

1. To design a normalised relational database (MySQL) for products, customers, orders, order items, expenses, staff and salaries, and to populate it with twelve months of realistic demo data.
2. To compute finance metrics — revenue, COGS, gross profit, operating expenses, net profit, margins, growth and cash flow — directly from transactional data.
3. To provide inventory management with stock status, inventory valuation and automatic low-stock alerts where current stock ≤ minimum stock.
4. To forecast product demand for the next three months and recommend stock levels, shortages and overstock.

5. To detect unusual expenses with a statistical method and explain why each one is unusual.
6. To compute a deterministic Financial Health Score out of 100 and explain every point.
7. To generate prioritised business recommendations from the analytics.
8. To provide a professional, responsive interface for staff and a customer panel for placing orders.

3. Review of Existing Approaches

SMEs today typically use manual registers and spreadsheets, or stand-alone accounting software. Spreadsheets need manual formulas and give no alerts; accounting software computes profit but rarely links it to stock, and neither forecasts demand, detects unusual expenses, scores overall financial health or suggests actions. The gap is therefore not the lack of data but the lack of interpretation: existing tools record transactions and leave the analysis to the owner. The proposed system turns the same records into explanations and actions.

4. Requirements Specification

4.1 Functional Requirements

Table 1: Functional requirements

ID	Requirement
FR1	Staff shall sign in with email and password; customers shall register and sign in separately.
FR3	Staff shall add, filter (category, month, text, unusual only), delete and export expenses.
FR4	The system shall compute monthly revenue, COGS, gross profit, operating expenses, net profit, margin and growth.
FR6	Staff shall view and filter all products with SKU, category, stock, minimum stock, prices, value, sales, supplier and status; add products and receive stock.
FR7	The system shall list products with current stock ≤ minimum stock, and out-of-stock and overstocked products separately.
FR9	The system shall forecast three months of demand per product with recommended stock, shortage and overstock.
FR11	The system shall compute the Financial Health Score and explain each factor.
FR13	Customers shall browse products, manage a cart, place orders and track them; staff shall update order status, and cancelling shall return stock.

4.2 Non-Functional Requirements

- **Accuracy and determinism:** every figure is calculated from database rows; the same data always produces the same score, forecast and recommendations.
- **Availability for demonstration:** if MySQL is not running, the application switches automatically to a local SQLite database with the same data.
- **Performance:** all analytics are computed once and cached until the data changes (a version counter is incremented on every write).
- **Security:** passwords are stored as salted PBKDF2-SHA256 hashes; staff and customer sessions are separate; all SQL uses parameterised queries.

- **Usability:** responsive layout, Indian number formatting (lakh / crore), status badges, filters and CSV export; Chart.js is bundled locally so the application works offline.

Software used: Python 3.10+, Flask 3, MySQL 8 (MySQL Workbench), SQLite fallback, scikit-learn, NumPy, Chart.js 4; runs on any PC with 4 GB RAM and a modern browser.

5. Proposed System and Architecture

The proposed system is a single web application with two user-facing parts — a **staff dashboard** with thirteen pages and a **customer panel** — sharing one backend and one database. It follows a three-tier architecture (Figure 1).

- **Presentation tier:** server-rendered Jinja2 pages styled with a custom CSS theme; Chart.js draws all charts in the browser.
- **Application tier:** Flask routes in app.py handle authentication, validation, page rendering and CSV export. The analytics and AI engine in analytics.py computes all metrics, forecasts, anomalies, the health score and recommendations.
- **Data tier:** db.py connects to MySQL, creates and seeds the database on first run, and falls back to SQLite when MySQL is unavailable.

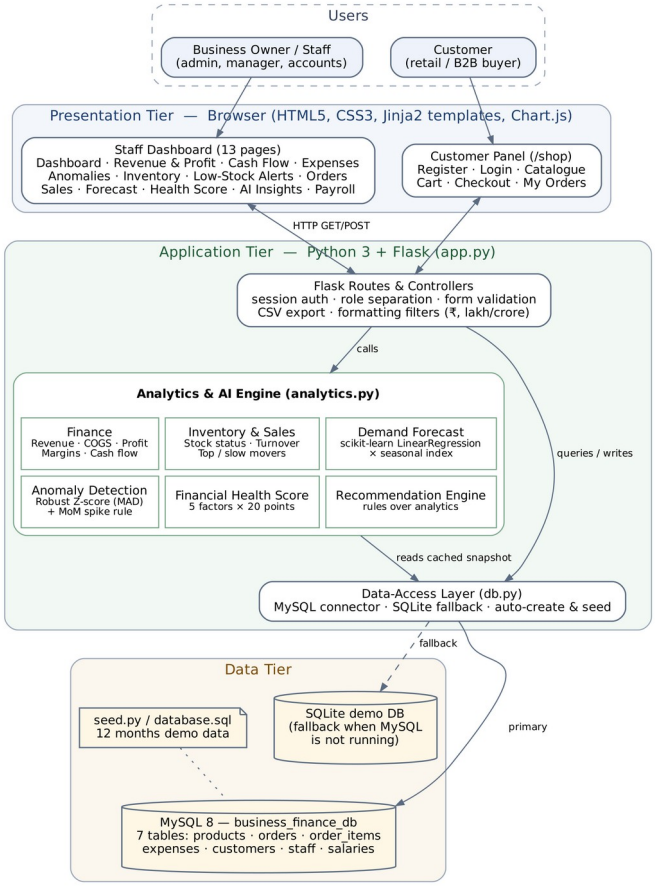


Figure 1: System Architecture

6. System Design (UML Diagrams)

6.1 Use Case and Activity Diagrams

The system has two actors. The **Staff / Admin** actor uses the finance, inventory, sales and AI features and manages customer orders. The **Customer** actor registers, browses products, manages a cart, places orders and tracks them. Placing an order includes managing the cart, and reviewing anomalies extends expense management. The activity diagram shows how a demand forecast and stock recommendation are produced for each product.

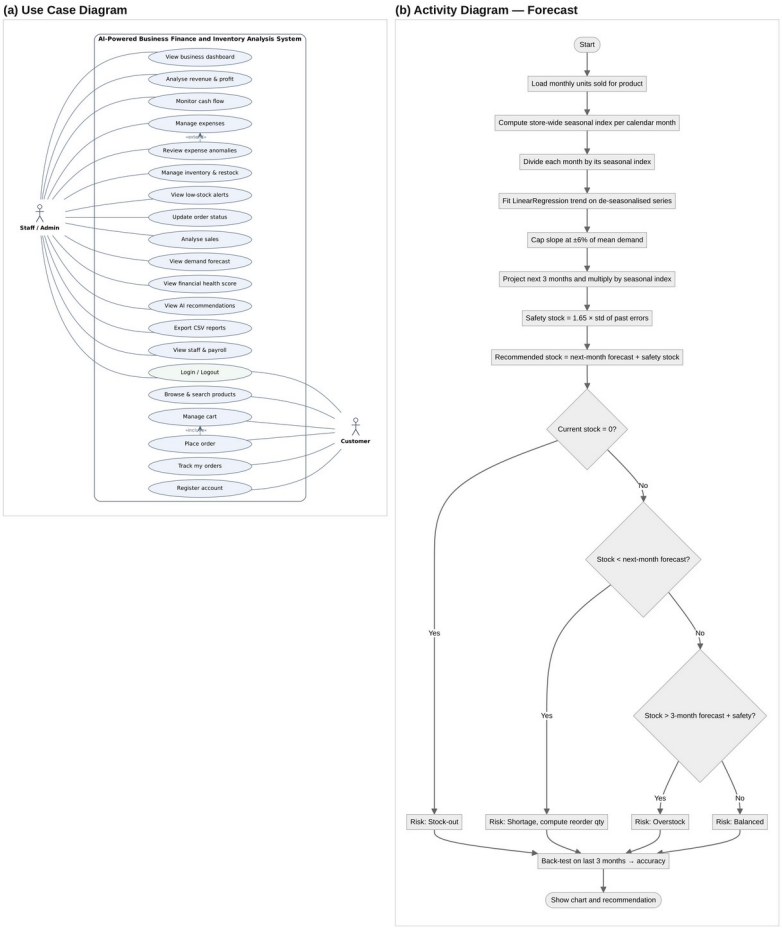


Figure 2: Use Case Diagram and Activity Diagram (Demand Forecast)

6.2 Class Diagram

The class diagram shows the main software components and the domain entities. FlaskApp uses the AnalyticsEngine and the DB data-access class; the engine loads a cached snapshot through DB; Seed generates demo data for an empty database. A Customer places Orders, each Order is composed of one or more OrderItems, and each OrderItem refers to a Product.

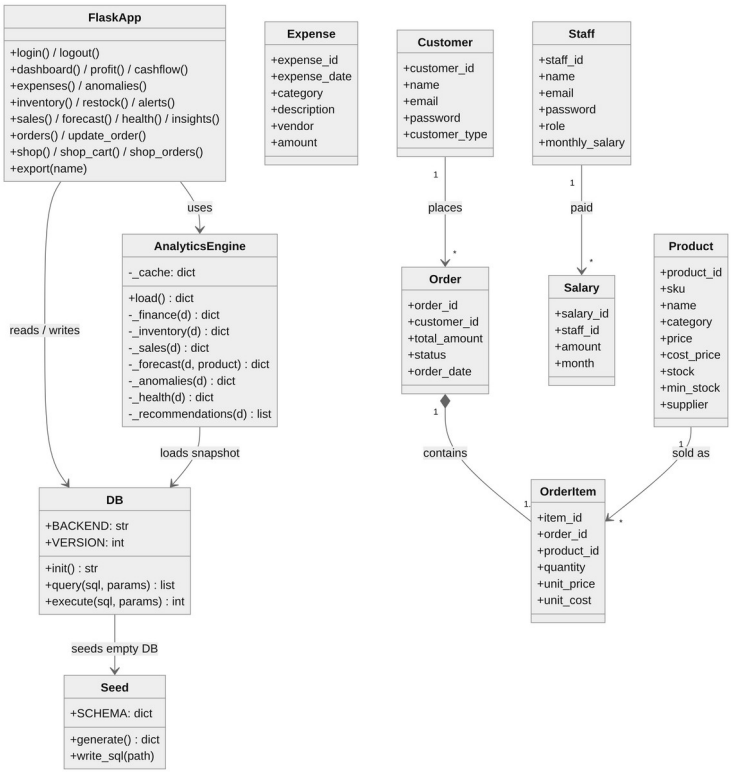


Figure 3: Class Diagram

6.3 Sequence Diagrams

The sequence diagram shows the customer placing an order. Stock is checked when a product is added to the cart and again at checkout; the order, its items and the stock reduction are written, and the next time staff open the alerts page the analytics snapshot is reloaded because the data version has changed.

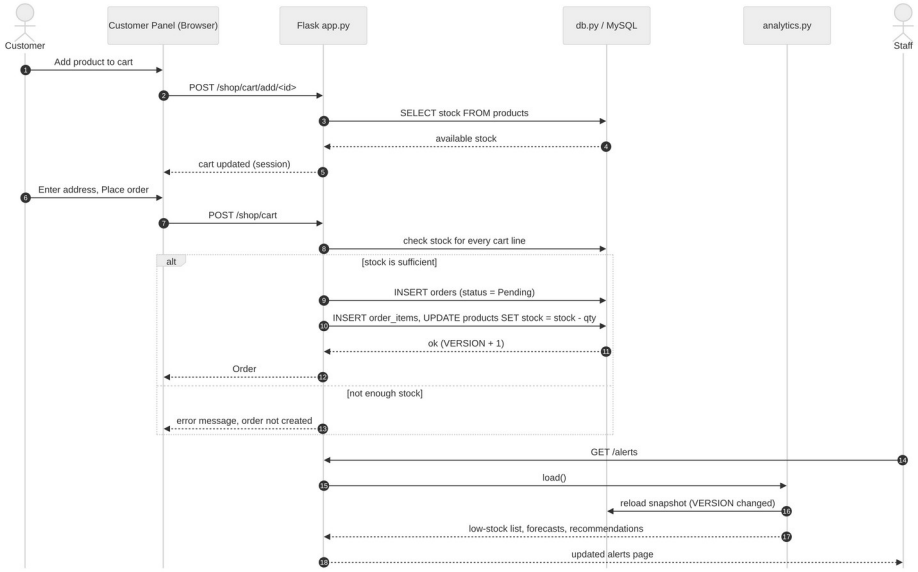


Figure 4: Sequence Diagram — Customer Places an Order

6.4 Entity–Relationship Diagram

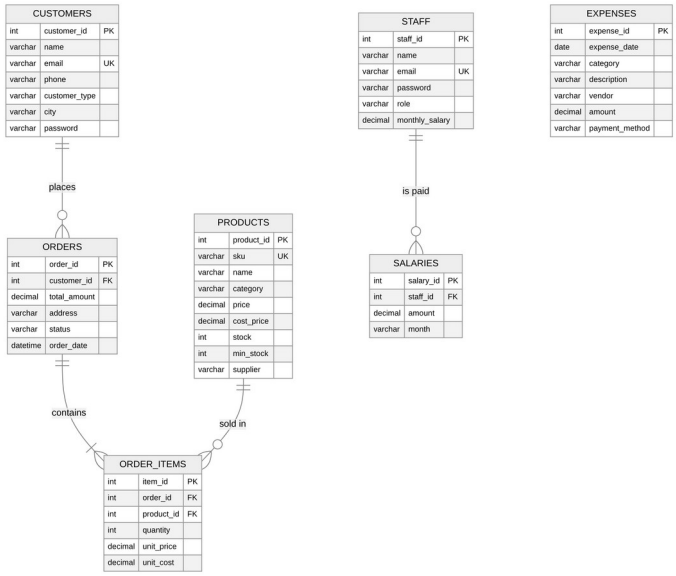


Figure 5: Entity–Relationship Diagram

7. Database Design and Dataset

7.1 Tables

The database business_finance_db contains seven tables in third normal form. Order items store the unit cost at the time of sale so that historical profit does not change when a product's cost price is updated later.

Table 2: Database tables

Table	Key columns	Purpose
products	product_id (PK), sku (UK), price, cost_price, stock, min_stock, supplier	Product master and current stock
customers	customer_id (PK), email (UK), customer_type, password	Retail and B2B customers; customer-panel login
orders	order_id (PK), customer_id (FK), total_amount, status, order_date	Sales orders: Pending → Processing → Shipped → Delivered / Cancelled
order_items	item_id (PK), order_id (FK), product_id (FK), quantity, unit_price, unit_cost	Order lines; source of revenue and COGS
expenses	expense_id (PK), expense_date, category, description, vendor, amount	All money paid out, in 8 categories
staff	staff_id (PK), email (UK), password, role, monthly_salary	Staff accounts for the dashboard
salaries	salary_id (PK), staff_id (FK), amount, month	Monthly payroll history

7.2 Demo Dataset

Because a real business's books could not be used, a deterministic generator (seed.py, fixed random seed 42) creates twelve months of data. The same file writes database.sql for import

into MySQL Workbench. The data intentionally contains festive-season peaks, growing and declining products, low-stock, out-of-stock and overstocked items, and four planted unusual expenses so that every feature has meaningful examples.

Table 3: Summary of the demo dataset

Property	Value	Property	Value
Period	Oct 2025 – Sep 2026	Products	24 in 8 categories
Sales orders	4,828 (4,736 not cancelled)	Order lines	9,448
Expenses	253 in 8 categories	Customers / staff	20 (6 B2B) / 6
Total revenue	₹2,07,41,820	Units sold	15,244
Average order value	₹4,380	Inventory value (cost)	₹14,45,460

Expenses fall into eight categories: Rent, Salaries, Utilities, Marketing, Inventory Purchase, Transportation, Operations and Miscellaneous.

8. Analytics and AI Methodology

8.1 Finance Metrics

Table 4: Finance formulas used

Metric	Formula
Revenue	Σ quantity × unit_price over non-cancelled orders
Cost of goods sold (COGS)	Σ quantity × unit_cost over non-cancelled orders
Gross profit / gross margin	Revenue − COGS; Gross profit ÷ Revenue × 100
Operating expenses	All expenses except Inventory Purchase
Net profit / net margin	Gross profit − Operating expenses; Net profit ÷ Revenue × 100
Cash inflow / outflow	Sales receipts; all expenses including stock purchases
Net cash flow	Inflow − Outflow (can be negative in a profitable month when stock is bought ahead)
Sales growth	Last 3 months' revenue vs the previous 3 months
Inventory turnover	Annual COGS ÷ inventory value at cost

Stock status uses the average daily demand of the last three months: **Out of Stock** when stock = 0, **Low Stock** when stock ≤ minimum stock, **Overstock** when stock covers more than 120 days, otherwise **In Stock**. A month that has only just started (less than 25% of a typical month's revenue) is shown separately as "in progress" so that a few new orders do not distort growth and forecasts.

8.2 Demand Forecasting

For each product the monthly units sold are divided by a store-wide seasonal index (each month's total units divided by the overall linear trend). A linear trend is fitted to the de-seasonalised series with scikit-learn's LinearRegression, the slope is limited to ±6% of average demand per month so that one unusual month cannot dominate, the trend is projected three months ahead and multiplied by the seasonal index of each target month.

- **Safety stock** = ⌈1.65 × standard deviation of the model's past errors⌉ (about a 95% service level).

- **Recommended stock** = next month's forecast + safety stock; **reorder quantity** = recommended stock − current stock.
- **Shortage** when stock < next month's forecast; **overstock** when stock > three months' forecast + safety stock.
- **Accuracy** is a back-test: the model is trained without the last three months, predicts them, and accuracy = 100% − weighted absolute percentage error.

8.3 Expense Anomaly Detection

Each operating expense is compared with similar expenses — those with the same description when at least six exist, otherwise the whole category. A robust Z-score is used because the mean and standard deviation are themselves distorted by the outliers being searched for:

z = (amount − median) ÷ (1.4826 × MAD), where MAD is the median absolute deviation.

An expense is flagged when z ≥ 3.5 **and** it is at least 1.5 times the typical (median) amount; it is High severity at 3 times or more, otherwise Medium. Separately, a category is flagged for a month when its total rose more than 30% over the previous month and is more than 30% above its typical month. Stock purchases are excluded because they follow sales volume.

8.4 Financial Health Score

Table 5: Financial Health Score factors (20 points each)

Factor	Measured value	Scoring rule
Profitability	Net profit margin	0 at break-even, full marks at 15% or more
Expense Control	Operating expenses ÷ revenue	Full marks at 20% or less, 0 at 40%
Cash Flow	Share of cash-positive months; last-3-month net cash ÷ inflow	10 points each; full at all months positive and at 10% of inflow
Revenue Growth	Last 3 months vs previous 3	0 at −10%, 10 at 0%, full at +10%
Inventory Efficiency	Turnover; share of products at healthy stock	10 points each; full at 8× a year and at 90% healthy

Each factor is scaled linearly between its zero and full-marks points and clipped. The total is graded Excellent (80+), Good (65–79), Fair (50–64) or Needs Attention (below 50). No random or hand-set values are used.

8.5 AI Recommendation Engine

The recommendation engine applies rules to the outputs above and ranks the results High, Medium or Low. Each recommendation quotes the figures that triggered it and links to the page with the details.

- **Inventory:** out of stock, stock lasting under 30 days, or more than 3 months of stock → reorder or clearance.
- **Sales:** units up or down by 10% or more over 3 months → add stock cover or review pricing.
- **Expenses:** category spike, unusual expenses, or costs growing faster than revenue → verify spend, set budgets.
- **Cash flow and profit:** negative or improved cash month, margin change of 1 point or more → stagger payments, review largest changes.

9. Implementation and Modules

The application consists of four Python files and twenty-two Jinja2 templates. The staff dashboard has thirteen pages: Dashboard, Revenue & Profit, Cash Flow, Expenses, Expense Anomalies, Inventory, Low Stock Alerts, Customer Orders, Sales Analytics, Demand Forecast, Financial Health, AI Recommendations and Staff & Payroll. The customer panel (/shop) provides registration, catalogue, cart, checkout and order tracking. Key screens are shown below.

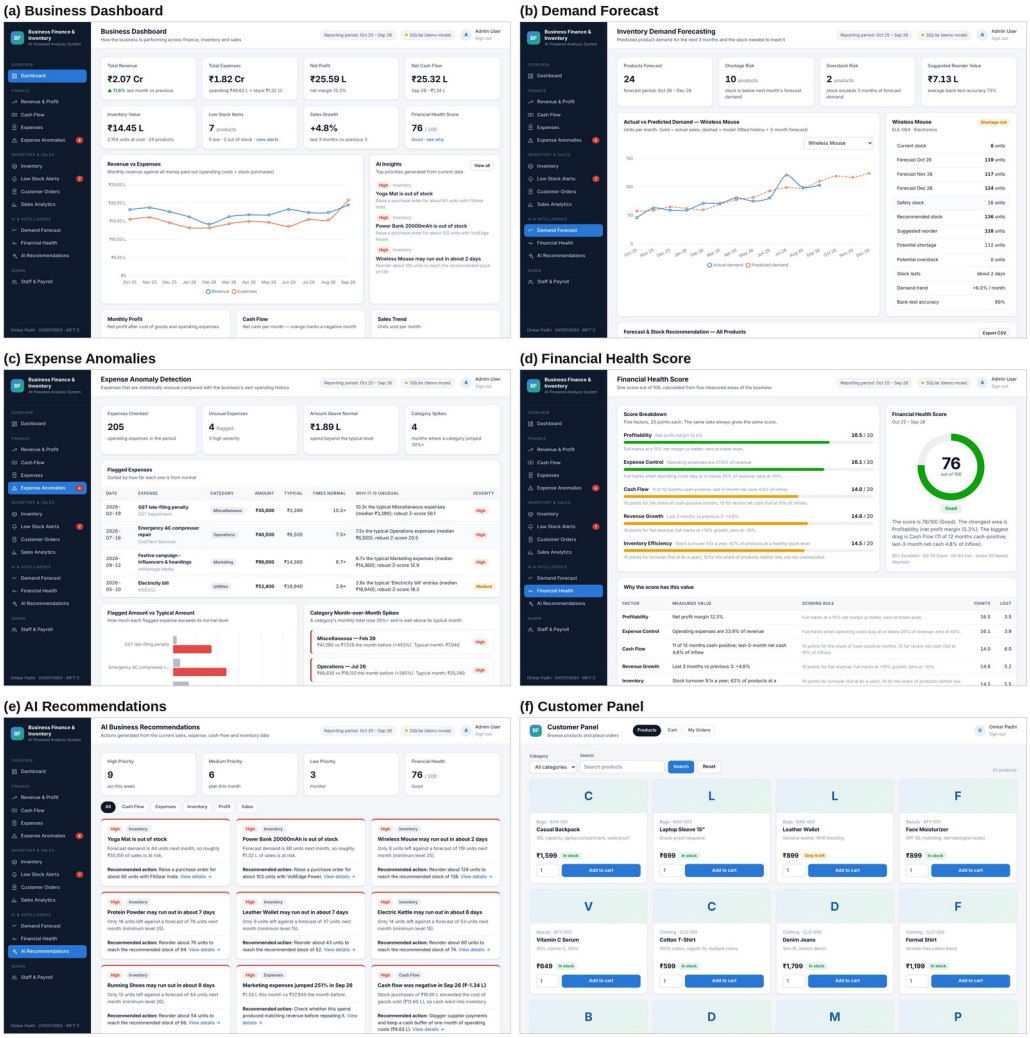


Figure 6: Key Screens of the Application

10. Testing

Automated end-to-end tests were written with Flask's test client and run against both the MySQL path (MariaDB server) and the SQLite fallback; the user interface was additionally checked in a headless Chromium browser for JavaScript errors.

Table 6: Test cases

#	Test case	Expected result	Result
1	Open a staff page without signing in	Redirect to the sign-in page	Pass
2	Staff sign-in with a wrong password	"Invalid email or password"	Pass
3	Add an expense of ₹2,50,000 in	Saved; flagged on the anomaly page	Pass

#	Test case	Expected result	Result
	Marketing		
4	Receive 5 units of stock	Stock increases by 5	Pass
5	Customer adds an out-of-stock product	“That product is out of stock”	Pass
6	Customer places an order for 2 units	Order created, stock reduced by 2	Pass
7	Staff cancels the order	Status Cancelled, 2 units returned to stock	Pass
8	New order in a new month	Shown as “in progress”; trend months unchanged	Pass
9	Start with MySQL stopped	Falls back to SQLite with the same data	Pass

11. Results and Discussion

11.1 Financial Results

Table 7: Monthly profit and cash flow (₹)

Month	Revenue	COGS	Operating exp.	Net profit	Margin	Net cash flow
Oct 25	18,22,742	11,48,250	4,07,429	2,67,063	14.7%	2,67,113
Nov 25	18,73,102	11,84,770	4,09,789	2,78,543	14.9%	2,66,613
Dec 25	17,62,292	11,19,420	4,10,689	2,32,183	13.2%	2,99,303
Jan 26	16,21,289	10,33,910	3,93,659	1,93,720	11.9%	2,97,130
Feb 26	14,21,903	9,00,190	4,24,969	96,744	6.8%	96,734
Mar 26	16,25,855	10,38,450	3,95,579	1,91,826	11.8%	1,91,776
Apr 26	16,78,891	10,75,650	3,91,559	2,11,682	12.6%	1,79,532
May 26	16,76,400	10,66,300	4,25,739	1,84,361	11.0%	2,05,761
Jun 26	18,27,013	11,69,160	3,90,989	2,66,864	14.6%	4,65,624
Jul 26	17,42,123	11,09,470	4,40,929	1,91,724	11.0%	1,91,694
Aug 26	17,42,086	11,15,390	3,87,839	2,38,857	13.7%	2,05,447
Sep 26	19,48,124	12,59,590	4,82,899	2,05,635	10.6%	−1,34,375
Total	**2,07,41,820**	**1,32,20,550**	**49,62,068**	**25,59,202**	**12.3%**	**25,32,352**

The business is profitable in every month, with a gross margin of 36.3% and a net margin of 12.3%. September 2026 shows the most important insight of the system: net profit was ₹2.06 lakh, but net cash flow was **−₹1.34 lakh** because stock purchases of ₹16.00 lakh were made ahead of the festive season. A profit-only report would hide this cash shortfall.

11.2 Inventory and Forecast Results

Of the 24 products, 15 are In Stock, 5 Low Stock, 2 Out of Stock (Yoga Mat, Power Bank 20000mAh) and 2 Overstocked (Desk Lamp LED, Laptop Sleeve 15″). Inventory turnover is 9.1 times a year. The forecast marks 10 products at shortage risk and 2 at overstock risk, with a suggested reorder value of ₹7.13 lakh. The average back-test accuracy is 73% (range 31% to 90%); products with steady demand such as Wireless Mouse (90%) forecast well, while products with sharply falling demand such as Desk Lamp LED and Laptop Sleeve forecast poorly.

11.3 Anomaly Detection Results

Table 8: Unusual expenses detected

Expense	Amount	Typical	Robust z	Severity
GST late-filing penalty (Feb 26)	₹35,000	₹3,390	56.1	High
Emergency AC compressor repair (Jul 26)	₹48,500	₹6,500	20.5	High
Festive campaign – influencers & hoardings (Sep 26)	₹96,000	₹14,360	12.9	High
Electricity bill (May 26)	₹52,400	₹18,940	18.3	Medium

All four expenses planted in the data were detected and no regular expense was flagged. The category-spike rule additionally flags the same four months (Miscellaneous in February, Utilities in May, Operations in July and Marketing in September).

11.4 Financial Health Score

Table 9: Financial Health Score on the demo data

Factor	Measured value	Points
Profitability	Net margin 12.3%	16.5 / 20
Expense Control	Operating expenses 23.9% of revenue	16.1 / 20
Cash Flow	11 of 12 months positive; last-3-month net cash 4.8% of inflow	14.0 / 20
Revenue Growth	+4.8% (last 3 months vs previous 3)	14.8 / 20
Inventory Efficiency	Turnover 9.1×; 62% of products at healthy stock	14.5 / 20
Total		**76 / 100 (Good)**

The engine produced 18 recommendations on this data (9 High, 6 Medium, 3 Low), for example: “Wireless Mouse may run out in about 2 days — only 8 units left against a forecast of 119 units next month; reorder about 128 units.”

11.5 Limitations

- The results are obtained on synthetic demo data generated by the project; they demonstrate the methods but are not measurements of a real business.
- The seasonal index is estimated from a single year of history, so forecasts should be treated as planning estimates; accuracy is low for products whose demand is falling sharply.
- The recommendation engine is rule-based; it does not learn from which recommendations the owner accepted.
- There is a single staff role level: every staff account sees all pages. Payment is cash on delivery only, and the system runs on a single machine.

12. Conclusion and Future Scope

12.1 Conclusion

The project transformed a basic e-commerce storefront into a working business-intelligence platform for SMEs. Sales, expenses and inventory are stored in one MySQL database and analysed together, so the owner can see profit, cash flow, stock risk and unusual spending on one dashboard. The forecasting, anomaly detection, health score and recommendation features are computed from the data with transparent, explainable rules and formulas, and every page was tested end to end on both the MySQL and SQLite back ends. The most useful result is the link between profit and cash: the system shows when a profitable month is still cash-negative and why.

12.2 Future Scope

- Role-based access for different staff roles (for example, sales staff limited to inventory and orders).
- Import of real bills and bank statements (CSV / Excel) and GST-ready reports.
- Forecasting with more history and more advanced models (for example, Prophet or gradient boosting) with weekly granularity.
- Automatic purchase-order generation and supplier lead-time tracking.
- Email or WhatsApp alerts for stock-outs and unusual expenses; online payments in the customer panel.
- Cloud deployment with HTTPS and scheduled database backups.

References

[1] J. Han, M. Kamber and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Morgan Kaufmann, 2011.

[2] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. OTexts, 2021. https://otexts.com/fpp3/

[3] B. Iglewicz and D. C. Hoaglin, How to Detect and Handle Outliers. ASQC Quality Press, 1993.

[4] P. J. Rousseeuw and C. Croux, “Alternatives to the Median Absolute Deviation,” Journal of the American Statistical Association, vol. 88, no. 424, pp. 1273–1283, 1993.

[5] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[6] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[7] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.

[8] Flask documentation, https://flask.palletsprojects.com; MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/; Chart.js documentation, https://www.chartjs.org/docs/.